

Performance Comparison of Open Speech-To-Text Engines using Sentence Transformer Similarity Check with the Korean Language by Foreigners

Aria Bisma Wahyutama*, Mintae Hwang†

Department of Information and Communication Engineering
Changwon National University
Changwon, Republic of Korea

*wahyutama@gs.cwnu.ac.kr, †mthwang@cwnu.ac.kr

Abstract—This paper contains the performance comparison of four Speech-to-Text (STT) engines which are Google STT, Naver Clova CSR, IBM Watson, and Microsoft Azure STT when transcribing foreigners speaking the Korean Language. The respondents are recording themselves speaking a predetermined sentence to be compiled together and then feeding it into the STT engine one by one to generate the transcribed text. The performance is evaluated using the Sentence Transformer Python framework that checks the similarity percentage between the original sentence to each of the transcribed texts and then finds the average result. The engine's performance is categorized into four different categories which are sentence, nationality, age, and gender. The performance comparison results can be used to help determine the optimal STT engine for the Korean Language Spoken by Foreigner to develop STT-based or AI-based applications.

Keywords—*Performance Comparison, Speech-to-Text, Sentence Transformer, Similarity Check, Korean Language, Optimal STT Engine*

I. INTRODUCTION

According to its official website [1], South Korea, which is officially named the Republic of Korea, is a country located in the middle of Northeast Asia, flanked by China to its West and Japan to its East. Being a country that is flanked by another country that does not use a Latin alphabet, South Korea has its language and alphabet system called 'Hangeul' that is composed of 14 consonant letters and 10 vowel letters. It was invented by King Sejong during the Joseon dynasty which made the Koreans very proud. Korean has a test to measure the foreigner's proficiency in the Korean language, called the Test of Proficiency In Korean (TOPIK) which is usually held six times a year inside South Korea and less often outside South Korea. TOPIK has six levels with level 1 being the lowest (beginner) and level 6 being the highest (advanced). Furthermore, TOPIK has become the requirement to get a job or be elected for a scholarship in South Korea.

TOPIK test measures proficiency in three fields, which are reading, writing, and listening comprehension in the Korean Language [2]. However, recently South Korean government has introduced a new field of test in TOPIK which is speaking. It is currently shown on [2] that the speaking test is conducted separately with the reading, writing, and listening. Furthermore, the speaking test is on a pilot operation meaning that it is still under trial and considered a new form of the

TOPIK test that would be implemented in the future if deemed to be successful. Currently, the scoring process of the speaking test in TOPIK is done manually by an expert panel, which will take a considerable amount of time especially when the speaking test is no longer a pilot project and the test participant is increasing.

The project in this paper is driven by NSDevil, which is a Korean company that has the authority to manage the TOPIK test across South Korea. The NSDevil has a plan to develop a system that can generate a foreigner's Korean language proficiency score automatically to be implemented in the TOPIK test to decrease the evaluation and scoring time. The system requires a Speech-to-Text (STT) engine to transcribe the test participant's voice into a text and feed it into a Machine or Deep Learning model to further examine the test participant's voice and generate the score automatically. The considerations of this system are how well the STT can transcribe the test participant's voice and how accurate the Machine Learning model is to generate a score because the speaking ability is based on transcribed text, holistic, segment, phonology, fluency, and pitch.

To support the ideal future system, this paper presents a performance evaluation comparison between four STT engines to test how well they transcribe recorded voices into a text as the first stage to develop the complete system. The transcribed results are then fed into a similarity checker to compare the similarity percentage between the transcribed text and the original text. Therefore, this paper can be used to provide help in deciding which engine is the most optimal to transcribe the Korean language spoken by Foreigners that will eventually achieve an ideal system to generate the Korean language speaking proficiency score automatically.

II. RELATED WORKS

The STT engine's main objective is to take audio as an input and generates text translation as an output. Producing an accurate result on transcribing speech into text is a prerequisite for all other natural language applications that operate on audio sources [3-4]. The STT engines became significantly popular in the 1990s when the performance of speech recognition algorithms reached a point where it provides decent speech transcripts, albeit it is not highly accurate. However, with the recent popularity of Deep Learning, STT engine performance is also increasing significantly to transcribe speech into text more accurately [5]. The general architecture of an STT engine is depicted in Fig. 1.

This work was supported by NSDevil Company and Gyeongnam SW Convergence Cluster 2.0, under contract.

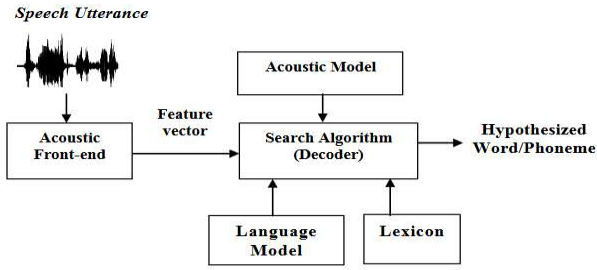


Fig. 1. General Architecture of Speech-to-Text Engine

There are several approaches in terms of algorithms to perform STT tasks as mentioned in [6], which are:

- Template-based approach: finding matching unknown speeches with already obtained pieces of information.
- Knowledge-based approach: a variation of speeches is stored in the system and a further complex rule embedded in the speech is deduced by an interference engine.
- Neural network-based approach: utilize Neural network AI algorithms to detect speech based on the training data automatically.
- Dynamic time warping-based approach: concentrate on matching sequences of the same speech that vary in time or speed.
- Statistical-based approach: utilize automatic learning procedures on a vast amount of training data. One of the most well-known algorithms with this approach is the Hidden Markov Model (HMM) which is the current state-of-the-art.

Although there are researches that attempted to develop an application to score foreigners' Korean language proficiency automatically, such as stated in [7-8], this paper only focuses on delivering the performance comparison between four different STT engines. Existing research regarding the comparison of multiple STT engines is also been done in [9-10]. In [9], the research only compares two STT engines which are Google STT and Kaldi to measure the recognition performance of each engine, while [10] compares manual transcription and online automatic transcription to check the performance for nonverbal response and unintelligible speeches. Therefore, based on the existed research, this paper's novelty is that the performance comparison is based on foreigners speaking the Korean language and it is conducted to help determine the optimal STT engine for developing the foreigner Korean language speaking proficiency auto-scoring application for the TOPIK test.

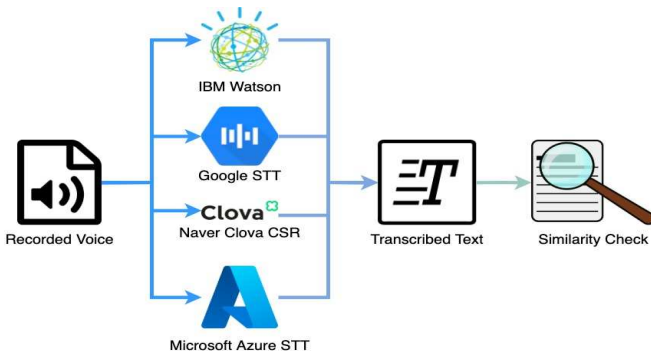


Fig. 2. Performance Measurement Method

TABLE I. SELECTED KOREAN SENTENCES

Selected Korean Sentences	Translated Sentences
내가 나를 사랑하지 않으면 세상 누구도 나를 사랑해 주지 않는다	If I don't love myself, nobody will love me in the world
좋은 사람을 만나고 싶으면 나부터 좋은 사람이 되어야 한다	If you want to meet a good person, you should be a good person first
사람들이 꿈을 이루지 못하는 이유는 생각을 바꾸지 않으면서 결과를 바꾸고 싶어하기 때문이다	The reason why people don't come true their dreams are because they want to change the results without changing their thinking

III. PERFORMANCE MEASUREMENT METHOD

A. Performance Method

The performance measurement method starts by selecting the reference sentence for the foreigners and then collecting the recorded voices. Once the gathered data is sufficient, each of the recorded voices is fed into all STT engines to generate the transcribed text. The transcribed results are then fed into a similarity check to measure how similar the transcribed text compared to the reference sentence with higher similarity indicates better performance. After the percentage results are obtained, they will be categorized into four categories (sentence, nationality, age, and gender) that will be shown in a form of a graph for easy reading. The performance measurement method is depicted in Fig. 2.

B. Sentence Selecting and Data Collecting

To maintain the fairness of the measurement, there are three selected Korean sentences for this case that would act as reference sentences for the foreigners. The three selected sentences as shown in Table I.

After determining the sentence, it was then distributed to foreigners around South Korea, specifically in Changwon, as well as Indonesia for a period from December 2021 to March 2022, for them to read and record themselves speaking the selected sentence. On a side note, there is no specific device and requirement of how they should record the voice.

Once all of the recorded voices are collected, the files are required to be converted to the same file extension and specification to facilitate the transcription process conducted by the STT engines. For the case in this paper, the audio files are converted to a WAV extension because it is an uncompressed audio extension. Due to its uncompressed audio extension, WAV is excellent in keeping the original quality of the audio as opposed to an MP3 extension which will reduce the original audio quality because it is a compressed audio extension. Whereas for the audio itself, there is no significant modification (e.g., noise reduction, vocal enhancement, et cetera) in any such to maintain the purity of the recorded voice and increase the variation of the data. Additionally, the recorded voices consist of various foreigners' proficiency levels from beginner to advanced. The audio specification is depicted in Table II.

TABLE II. AUDIO SPECIFICATION

Parameter	Specification
Bit Resolution	16-bit
Audio Frequency	44100Hz / 44.1kHz
Audio Channel	1 / Mono

TABLE III. STT ENGINES DESCRIPTION

STT Engines	Description
Google STT	- Familiar with rare words or phrase - Automatically convert spoken numbers. - Provides model customization to enable experimentation, creation, and management of custom resources
IBM Watson	- Provides a model that is optimized for low latency in real-time speech applications. - Currency, values, email, dates, numbers, times, and website address can be transcribed into normal readable text - An audio signal is analysed and then corrected accordingly before begin the transcription begins
Naver Clova CSR	- An STT engine with the highest accuracy for the Korean language - Multilingual support including English, Japanese, and Chinese (simplified) - Provides a Mobile SDK and REST API to make the service available in a different environment across mobile devices and servers.
Microsoft Azure STT	- Get accurate audio to text transcriptions with state-of-the-art speech recognition. - Add specific words to your base vocabulary or build custom speech-to-text models. - Access the same robust technology that powers speech recognition across Microsoft products.

For this research, the recorded voices are coming from 100 people with each person speaking three sentences, making it a total of 300 recorded voices that includes foreigners from Indonesia, Iran, Mongolia, Nepal, Philippines, and Vietnam. The foreigner's ages are ranging from 14 to 51 years old and the gender distribution is 32 males and 68 females.

C. Speech to Text Engine

The STT engines are chosen based on several considerations, such as all of them must be open-source, provide user-friendly documentation, has multiple pricing plans for different scenarios, relatively well-known to the public, supports to be developed in multiple programming languages, and supports the Korean language. The key features of each selected STT engine, as compiled from their official websites, are shown in Table III.

Before properly running the STT engine, they required several things. First is an access ID that can be obtained by creating an account as a developer on their official websites. Once the account is created, access the developer dashboard to register STT services and generate the access ID. The access ID later will be written directly onto the code to start the STT engine. The second is an external library. All except one (Naver Clova CSR) require an external library that is needed to be installed. Due to this paper uses a Python programming language in Jupyter Notebook with an ipynb (Interactive PYTHON Notebook) extension, therefore, installing the required library can be done via the pip install command, which are 'pip install --upgrade google-cloud-speech', 'pip install azure-cognitiveservices-speech', and 'pip install ibm_watson' for Google API, Microsoft Azure, and IBM Watson, respectively.

The code to execute the STT engine can be found in their documentation and is rather easy to use. All the primary codes for each engine are shown in Table IV.

TABLE IV. STT ENGINE SOURCE CODE

Google STT
<pre># Import internal library import io import os # Imports the Google Cloud client and audio library from google.cloud import speech from IPython.display import Audio os.environ['GOOGLE_APPLICATION_CREDENTIALS'] = 'ACCESS_CREDENTIAL_JSON_PATH.json' # Instantiates a client client = speech.SpeechClient() # The name of the audio file to transcribe gcs_uri = "AUDIO_FILE_PATH.wav" audio = speech.RecognitionAudio(uri=gcs_uri) config = speech.RecognitionConfig(encoding=speech.RecognitionConfig.AudioEncoding.LINEAR16, sample_rate_hertz=16000, language_code="en-US",) # Detects speech in the audio file response = client.recognize(config=config, audio=audio) for result in response.results: print("Transcript: {}".format(result.alternatives[0].transcript))</pre>
Naver Clova CSR
<pre># Import library import json import requests # Load audio file data = open("AUDIO_FILE_PATH.wav", "rb") # Language selection Lang = "Kor" URL = "https://naveropenapi.apigw.ntruss.com/recog/v1/stt?lang=" + Lang # Client and Secret ID ID = "CLIENT_ID" Secret = "SECRET_CODE" headers = { "Content-Type": "application/octet-stream", "X-NCP-APIGW-API-KEY-ID": ID, "X-NCP-APIGW-API-KEY": Secret, } response = requests.post(URL, data=data, headers=headers) rescode = response.status_code if(rescode == 200): print(response.text) else: print("Error : " + response.text)</pre>
IBM Watson
<pre># Import IBM Watson library from ibm_watson import SpeechToTextV1 from ibm_watson.websocket import RecognizeCallback, AudioSource from ibm_cloud_sdk_core.authenticators import IAMAuthenticator #API key and URL apikey = 'API_KEY' url = 'STT_URL' authenticator = IAMAuthenticator(apikey) stt = SpeechToTextV1(authenticator=authenticator) stt.set_service_url(url) with open('AUDIO_FILE_PATH.wav', 'rb') as f: res = stt.recognize(audio=f, content_type='audio/wav', model='ko-KR_NarrowbandModel') .get_result() print((res['results'][0]['alternatives'][0]['transcript'])); print(res['results'][0]['alternatives'][0]['confidence']);</pre>
Microsoft Azure STT
<pre>#Import Microsoft Azure library import azure.cognitiveservices.speech as speechsdk #Subscription and region ID speech_config = speechsdk.SpeechConfig(subscription="dfc3ec41fa74904b03b1862b5a01b7a", region="koreacentral") speech_recognizer = speechsdk.SpeechRecognizer(speech_config=speech_config, audio_config=audio_config) #Load the audio file audio_config = speechsdk.audio.AudioConfig(filename="AUDIO_FILE_PATH.wav") speech_recognizer = speechsdk.SpeechRecognizer(speech_config=speech_config, audio_config=audio_config) speech_recognition_result = speech_recognizer.recognize_once_async().get() if speech_recognition_result.reason == speechsdk.ResultReason.RecognizedSpeech: print("Speech recognition result: {}".format(speech_recognition_result.text)) elif speech_recognition_result.reason == speechsdk.ResultReason.NoMatch: print("No speech could be recognized: {}".format(speech_recognition_result.no_match_details)) elif speech_recognition_result.reason == speechsdk.ResultReason.Canceled: cancellation_details = speech_recognition_result.cancellation_details print("Speech Recognition canceled: {}".format(cancellation_details.reason)) if cancellation_details.reason == speechsdk.CancellationReason.Error: print("Error details: {}".format(cancellation_details.error_details))</pre>


```

# Import Sentence Transformer and NumPy library
from sentence_transformers import SentenceTransformer, util
import numpy as np

# Embedd KoBERT model
embedder = SentenceTransformer("jhgan/ko-sbert-sts")

# Corpus with example sentences
corpus = ['TRANSCRIBED_SENTENCE_FROM_STT_ENGINE']

corpus_embeddings = embedder.encode(corpus, convert_to_tensor=True)

# Query sentences:
queries = ['ORIGINAL_REFERENCE_SENTENCE']

# Define how many sentences in corpus array.
top_k = 3
for query in queries:
    query_embedding = embedder.encode(query, convert_to_tensor=True)
    cos_scores = util.pytorch_cos_sim(query_embedding, corpus_embeddings)[0]
    cos_scores = cos_scores.cpu()

    for idx in range(top_k):
        print("%.4f" % (cos_scores[idx])) # Print the similarity percentage

```

Fig. 3. Source Code for Similarity Check

D. Similarity Check

The similarity check is done by using the Sentence Transformers which is a Python framework for state-of-the-art sentence, text, and image embeddings that are based on PyTorch and Transformers. It offers a large collection of pre-trained models tuned for various tasks, including comparing Korean language sentences. The source code in this paper is based on [11], which is a project that was developed to make the KoBERT model easier to use in the Sentence Transformer framework. KoBERT is a model that is a result of pre-training language representation which was developed specifically for the Korean language using the BERT method provided by Google to be used in various Natural Language Processing tasks.

Using the provided source code allows the usage of the uploaded KoBERT model in Hugging Face Hub. Hugging Face Hub is a website where it contains multiple Machine Learning and Deep Learning model for various purposes. Therefore, following the source code provided in [11], the transcribed text from the STT engine can be successfully compared its similarity.

An external library is required before executing the code to perform the similarity check. Installing the library can be done using the pip command, which is 'pip install ko-sentence-transformers'. The code to perform a similarity check is shown in Fig. 3. The model used for the Sentence Transformers framework is 'jhgan/ko-sbert-sts' developed by Junghyun Gan [11]. The 'sts' in the model's name stands for Semantic Text Similarity to assign a score on the similarity between two texts and the 'ko' in the model's name indicates that the model is developed for the Korean language.

IV. PERFORMANCE COMPARISON

After the whole performance evaluation method has been completed, the average similarity results score is calculated and then put into four different charts to show the results in four different categories as mentioned previously, which are sentence, nationality, age, and gender. For sentence category, the graph is depicted in Fig. 4.

Based on Fig. 4, Google STT has the highest similarity score with an average of 84.17%, followed by Naver Clova CSR with an average of 75.42%, Microsoft Azure STT with an average of 73.65%, and IBM Watson with an average of 40.55%.

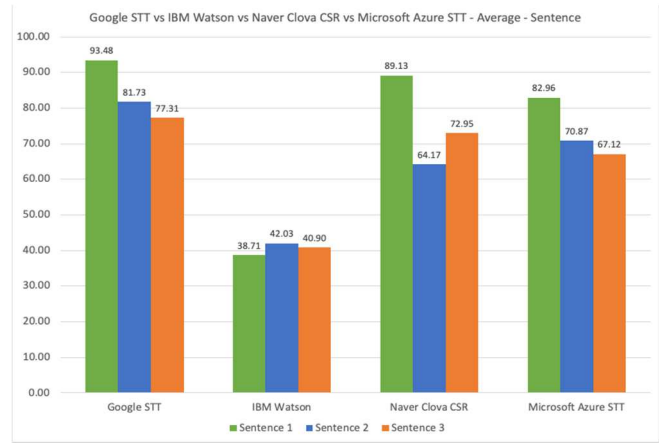


Fig. 4. Sentence Category Performance Graph

For the nationality category, the graph is depicted in Fig. 5. In the nationality category, a similar result is produced with the Google STT it still resulted in the highest similarity score with an average of 87.30%, followed by Naver Clova CSR with an average of 82.11%, Microsoft Azure STT with an average of 75.80%, and the IBM Watson with an average of 51.85%.

For the age category, the graph is depicted in Fig. 6 and it shows a similar result. Google STT is the best among other engines with an average result of 86.10%, while Naver Clova CSR averaged 79.35%, Microsoft Azure STT in third place with an average of 73.55%, and IBM Watson in the fourth place with an average of 40.43%.

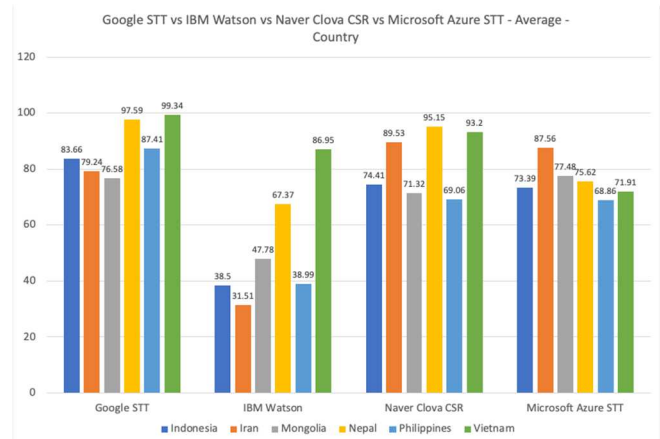


Fig. 5. Nationality Category Performance Graph

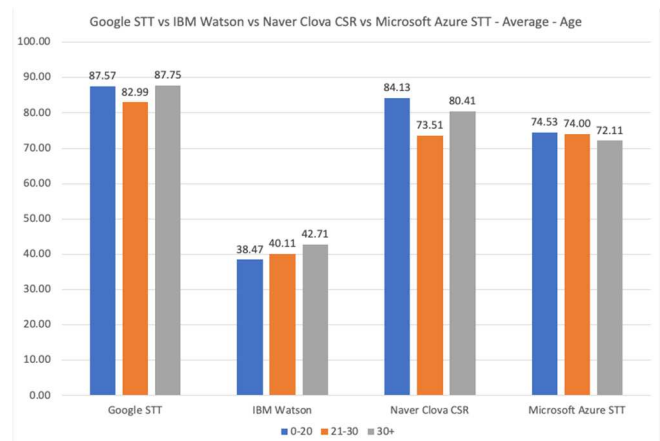


Fig. 6. Age Category Performance Graph

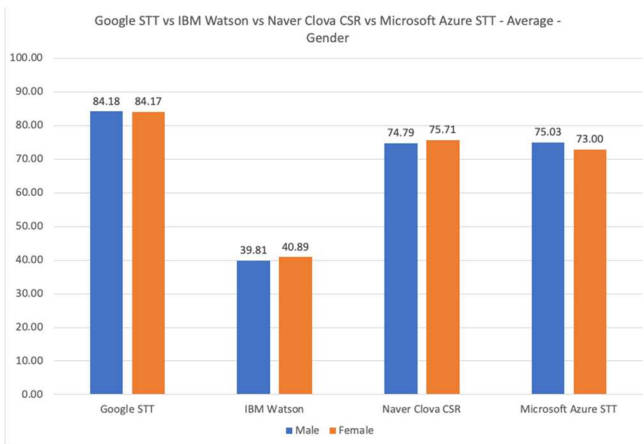


Fig. 7. Gender Category Performance Graph

For the age category, the graph is depicted in Fig. 7. In the age category, the engine's performance is the same as the previous three categories, which the Google STT consistently performed the best with an average similarity of 84.17%, followed by Naver Clova CSR with 75.25% average similarity, then Microsoft Azure STT with an average similarity of 74.02%, and IBM Watson in the fourth place with an average of 40.35%.

Based on the measured performance, Google STT performed the best in every category, with Naver Clova CSR in second place, Microsoft Azure STT in third place, and IBM Watson in fourth place. Although it might produce different results if some tweaks are applied to the engines, this paper only shows the basic performance before applying any tweak which acts as a baseline or benchmark, thus see how well they are performed and capable of handling a task to transcribe the foreigner's recorded voice speaking Korean sentences.

V. CONCLUSION AND FUTURE STUDIES

This paper shows the performance comparison between four well-developed and commercialized STT engines, which are Google STT, IBM Watson, Naver Clova CSR, and Microsoft Azure STT. The performance measurement starts by collecting the recorded voice of a foreigner speaking a selected Korean sentence and then feeding them into all of the STT engines. Once the voice is transcribed into text, the results are then fed into a similarity checker program that uses the SentenceTransformer framework and the language model that is optimized for the Korean language to check how similar the transcribed text compared to the original sentence in percentage value. Higher similarity indicates better performance of the engine.

After running the performance measurement, it is concluded that the Google STT is the best open-source STT engine compared to other engines presented in this paper with a consideration that it still performed well and consistent regardless of the spoken sentences, nationality, age, and gender. Therefore, based on the results, this paper highly recommends Google STT as the open-source STT engine to be used in the development of the Korean language speaking

proficiency auto-scoring system that can be implemented in the TOPIK test or any general purposes.

There are several research activities for future studies. The primary activity is to collect more voices that speak different sentences to further thoroughly examine the performance, including adding more test standards for resulting in more objective and valid results. Another activity for the future study is that the voice, transcribed text, and similarity score pair resulting in this paper will be used to develop a Machine Learning-based score prediction model that will generate a score without the need to go through a similarity check. Once the model has been fully developed, it will be implemented in a mobile or web application to automatically generate a score of how well the spoken Korean sentences based on the voice recording for multiple purposes, such as the TOPIK test, students that learn Korean by self-taught, teaching aid for school teacher, or employee candidate test in a certain organization that requires them to fluently speaks Korean.

REFERENCES

- [1] K. C. and I. Service (KOCIS), 'South Korea – Summary : Korea.net : The official website of the Republic of Korea'. <https://www.korea.net/AboutKorea/Society/South-Korea-Summary> (accessed Mar. 26, 2022).
- [2] 'TOPIK Korean Proficiency Test'. <https://www.topik.go.kr/TWMAIN/TWMAIN0010.do> (accessed Mar. 26, 2022).
- [3] S. Indurthi et al., 'End-end Speech-to-Text Translation with Modality Agnostic Meta-Learning', in ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), May 2020, pp. 7904–7908. doi: 10.1109/ICASSP40776.2020.9054759.
- [4] A. Stolcke et al., 'Recent innovations in speech-to-text transcription at SRI-ICSI-UW', IEEE Transactions on Audio, Speech, and Language Processing, vol. 14, no. 5, pp. 1729–1744, Sep. 2006, doi: 10.1109/TASL.2006.879807.
- [5] K. Ziman, A. C. Heusser, P. C. Fitzpatrick, C. E. Field, and J. R. Manning, 'Is automatic speech-to-text transcription ready for use in psychological experiments?', Behav Res, vol. 50, no. 6, pp. 2597–2605, Dec. 2018, doi: 10.3758/s13428-018-1037-4.
- [6] B. Iancu, 'Evaluating Google Speech-to-Text API's Performance for Romanian e-Learning Resources', IE, vol. 23, no. 1/2019, pp. 17–25, Mar. 2019, doi: 10.12948/issn14531305/23.1.2019.02.
- [7] Y. R. Oh, K. Park, H.-B. Jeon, and J. G. Park, 'Automatic proficiency assessment of Korean speech read aloud by non-natives using bidirectional LSTM-based speech recognition', ETRI Journal, vol. 42, no. 5, pp. 761–772, 2020, doi: 10.4218/etrij.2019-0400.
- [8] J.-U. Bang et al., 'KsponSpeech: Korean Spontaneous Speech Corpus for Automatic Speech Recognition', Applied Sciences, vol. 10, no. 19, Art. no. 19, Jan. 2020, doi: 10.3390/app10196936.
- [9] T. Kimura, T. Nose, S. Hirooka, Y. Chiba, and A. Ito, 'Comparison of Speech Recognition Performance Between Kaldi and Google Cloud Speech API', in Recent Advances in Intelligent Information Hiding and Multimedia Signal Processing, Cham, 2019, pp. 109–115. doi: 10.1007/978-3-030-03748-2_13.
- [10] J. Y. Kim et al., 'A Comparison of Online Automatic Speech Recognition Systems and the Nonverbal Responses to Unintelligible Speech', arXiv:1904.12403 [cs, eess], Apr. 2019, Accessed: Mar. 26, 2022. [Online]. Available: <http://arxiv.org/abs/1904.12403>
- [11] 'jhgan/ko-sbert-sts · Hugging Face'. <https://huggingface.co/jhgan/ko-sbert-sts> (accessed Mar. 26, 2022).